

Matriz de principios de diseño vs. estilo Hexagonal

Sección del documento técnico. Evalúa cómo el estilo **Hexagonal (Puertos y Adaptadores)** favorece o dificulta cada principio, con **ejemplos reales del código** del backend (rama del taller, septiembre de 2026). Cada ejemplo enlaza al archivo y la línea.

Escala:  se cumple ·  se cumple con excepciones conocidas ·  no se cumple

Matriz resumen

Principio	Cómo lo favorece Hexagonal	Cumplimiento	Ejemplo real en el código
S — Responsabilidad única	Separa núcleo (reglas) de periferia (HTTP, BD); cada adaptador tiene una razón para cambiar.		Un caso de uso por clase: <code>Postular</code> , <code>AprobarPostulacion</code> , <code>RechazarPostulacion</code> (postulaciones.py). <code>jsonapi.py</code> solo traduce; <code>errors.py</code> solo mapea errores.
O — Abierto/cerrado	Nueva tecnología = nuevo adaptador, sin tocar el núcleo.		Una excepción de dominio nueva hereda el código HTTP de su padre sin modificar el manejador (errors.py:63).
L — Sustitución de Liskov	Todo adaptador que cumple el puerto es intercambiable.		Los mismos casos de uso corren con el adaptador in-memory, con mocks y (próximamente) con Mongo, sin cambiar una línea (test_ports.py).
I — Segregación de interfaces	Un puerto por necesidad del núcleo, no un repositorio genérico.		Cuatro puertos pequeños; <code>RefugioRepository</code> solo tiene <code>obtener</code> y <code>listar</code> (repositories.py:72).
D — Inversión de dependencias	Es el corazón del estilo: el núcleo define las interfaces; la infraestructura las implementa.		Los casos de uso reciben <code>AnimalRepository</code> (un <code>Protocol</code>); el concreto lo decide <code>dependencies.py</code> con <code>Depends()</code> (dependencies.py:43).
KISS	Neutral: el estilo añade capas; la simplicidad hay que cuidarla dentro de cada una.		Aprobar o rechazar se decide en una línea (postulaciones.py:91). Costo: más archivos que un CRUD (ver matriz de calidad).
DRY	Las reglas viven en un solo lugar (el dominio), no en cada canal de entrada.		Una sola tabla excepción → HTTP (<code>MAPEO_DOMINIO</code> , errors.py:26). Excepción deliberada: la validación de formato se repite en el frontend.

Principio	Cómo lo favorece Hexagonal	Cumplimiento	Ejemplo real en el código
YAGNI	Los puertos solo declaran lo que un caso de uso usa hoy.	✓	Sin login, sin ORM, sin repositorio genérico, sin transacciones: nada de eso lo pide el taller.
PoLA — Menor sorpresa	Contratos explícitos (puertos, JSON:API) hacen predecible el comportamiento.	⚠	Los repositorios devuelven copias, como una BD real (memory/repositories.py:30). Sorpresa conocida: <code>POST /adoptantes</code> responde 200 si el email ya existe.
Ley de Demeter	El caso de uso habla con puertos y entidades, no con sus entrañas.	⚠	<code>animal.validar_postulable()</code> en vez de revisar <code>animal.estado</code> desde fuera (entities.py:118). Violación: <code>cuerpo.data.relationships.animal.data.id</code> en el router.
Composición sobre herencia	Los adaptadores se <i>componen</i> en los casos de uso; no se heredan.	⚠	Los casos de uso reciben sus puertos por constructor. Excepción: la clase base <code>_ConRepositorios</code> reutiliza código por herencia (postulaciones.py:18).
STUPID (anti-principios)	Hexagonal ataca sobre todo el acoplamiento fuerte y la baja testabilidad.	⚠	4 de 6 evitados. Quedan dos <i>singletons</i> controlados (cliente de Mongo y <code>get_repositorios</code>) y un acoplamiento entre adaptadores in-memory.

Detalle por principio

SOLID

S — Responsabilidad única. Cada caso de uso es una clase con un solo método público `ejecutar()`. Si cambia la regla de cierre automático, solo cambia `AprobarPostulacion`; si cambia el formato JSON:API, solo cambia `jsonapi.py`; si cambia un código HTTP, solo cambia `errors.py`. El dominio no sabe de ninguno de los tres.

O — Abierto/cerrado. El manejador de errores recorre la jerarquía de la excepción (`__mro__`) buscando un código:

```
for tipo in type(exc).__mro__:          # errors.py:63
    if tipo in MAPEO_DOMINIO:
        status, code, title, pointer = MAPEO_DOMINIO[tipo]
        break
else:
    status, code, title, pointer = 409, "REGLA_DE_NEGOCIO", ...
```

Agregar `class AnimalEnCuarentena(AnimalNoDisponible)` responde 409 `ANIMAL_NO_DISPONIBLE` sin tocar el manejador, y una regla nueva sin mapeo responde 409 genérico, nunca 500 (probado en `test_errores_http.py`). A escala del estilo: el adaptador de Mongo se agrega sin modificar casos de uso ni routers.

L — Sustitución de Liskov. Cualquier clase que cumpla el `Protocol` sustituye a otra sin que el caso de uso lo note. La suite lo demuestra con tres sustitutos del mismo puerto: el adaptador in-memory (`test_use_cases.py`), mocks `AsyncMock(spec=AnimalRepository)` (`test_casos_de_uso_con_mock.py`) y *fakes* mínimos (`test_ports.py`). El adaptador in-memory incluso devuelve **copias** para no ofrecer garantías más débiles que una base real: si devolviera la misma instancia, un caso de uso que olvida llamar a `guardar()` pasaría las pruebas y fallaría con Mongo.

I — Segregación de interfaces. En vez de un `Repository[T]` genérico con `crear/leer/actualizar/borrar/buscar` , hay cuatro puertos con lo que cada caso de uso necesita:

Puerto	Métodos	Por qué no más
<code>AnimalRepository</code>	<code>obtener</code> , <code>listar</code> , <code>guardar</code>	Nadie borra animales.
<code>PostulacionRepository</code>	<code>obtener</code> , <code>listar_por_animal/adoptante/refugio</code> , <code>existe_pendiente</code> , <code>guardar</code>	<code>existe_pendiente</code> existe solo para la regla 2.
<code>RefugioRepository</code>	<code>obtener</code> , <code>listar</code>	Los refugios no se crean desde la API.
<code>AdoptanteRepository</code>	<code>obtener</code> , <code>obtener_por_email</code> , <code>guardar</code>	<code>obtener_por_email</code> sirve al registro sin login.

D — Inversión de dependencias. Los casos de uso importan `AnimalRepository` de `application/ports` , nunca `InMemoryAnimalRepository` ni Motor. El único archivo que conoce las implementaciones concretas es `dependencies.py` ; los routers piden Repositorios con `Depends(get_repositorios)` y las pruebas lo reemplazan con `app.dependency_overrides` . Una prueba verifica automáticamente que el dominio no importe infraestructura (`test_el_dominio_no_importa_infraestructura`).

KISS — Keep It Simple

- El adaptador in-memory es un diccionario por entidad (`memory/repositories.py:24`).
- El `PATCH` elige el caso de uso con una sola línea, sin patrón *strategy* ni fábricas (`postulaciones.py:91`).
- Los estados son `StrEnum` , no una máquina de estados con librería externa: las transiciones son métodos de la entidad (`registrar_postulacion` , `marcar_adoptado` , `liberar`).

```
caso = AprobarPostulacion if cuerpo.data.attributes.estado == "aprobada" else
RechazarPostulacion
```

Tensión con el estilo: Hexagonal añade puertos, adaptadores y mapeos. Para 4 entidades es más estructura que un CRUD; se justifica por las reglas de estado (ver matriz de calidad, fila *Simplicidad*).

DRY — Don't Repeat Yourself

- **Una sola tabla** de excepción de dominio → HTTP (`MAPEO_DOMINIO`), usada por todos los endpoints.
- **Una sola implementación** del cambio de estado de una postulación: `aprobar()` , `rechazar()` y `cerrar_automáticamente()` delegan en `_resolver()` (`entities.py:161`).
- **Una sola base** para los cuatro repositorios en memoria (`_Almacen` , copias e ids).
- **Repetición deliberada:** la validación de formato (email, teléfono, longitud del mensaje) existe en el frontend (feedback inmediato) y en el backend (fuente de verdad). Es la misma regla en dos

procesos distintos, no código duplicado dentro de uno.

YAGNI — You Aren't Gonna Need It

Lo que **no** se construyó porque el taller no lo pide: autenticación, ORM/ODM (Beanie aparece en el ADR-007 pero no fue necesario), repositorio genérico, *unit of work* y transacciones, eventos de dominio, caché. Los puertos declaran solo métodos que algún caso de uso llama hoy; por ejemplo, no hay `borrar()` en ningún puerto.

Frontera honesta: los `links` de paginación (`first/prev/next/last`) los exige JSON:API pero el frontend no los usa; se mantienen por el contrato, no por especulación.

PoLA — Principio de menor sorpresa

- **Copias en memoria:** modificar una entidad no cambia lo guardado hasta llamar a `guardar()` , igual que con una base de datos.
- **Estado de solo lectura:** `POST /animals` ignora un `estado` enviado por el cliente; el animal siempre nace `disponible` (documentado en el contrato).
- **Errores predecibles:** todo error, incluso 404 de ruta o 500, sale en formato JSON:API con `code` ; nunca HTML ni texto plano.
- **Mensajes en el idioma y el género correctos:** "Luna ya fue adoptada." ([entities.py:110](#)).
- **Sorpresa conocida:** `POST /adoptantes` con un email existente responde **200** con ese adoptante en vez de 201 o 409. Es una decisión de registro sin login (contrato, sección 5.3), documentada para que no sorprenda.

Ley de Demeter — "habla solo con tus amigos"

Se cumple en el núcleo. El caso de uso le *pide* a la entidad que se valide, en vez de inspeccionar su estado:

```
animal.validar_postulable()      # Postular (en lugar de: if animal.estado == "adoptado": ...)
postulacion.aprobar()           # la entidad sabe qué campos cambiar
```

Violaciones conocidas (en la periferia): - `cuerpo.data.relationships.animal.data.id` en el router de postulaciones: una cadena de 5 accesos. Es la forma del documento JSON:API; se contiene en el adaptador de entrada y no llega al dominio. Mejora posible: un método `cuerpo.animal_id()` en el esquema, como ya existe `a_entidad()` . - `r.direccion.ciudad` al serializar un refugio ([jsonapi.py:62](#)): acceso a un objeto de valor anidado, aceptable.

Composición sobre herencia

- **Composición (lo predominante):** los casos de uso *tienen* repositorios (inyectados por constructor), no heredan de ellos; `Animal` se compone de objetos de valor `Salud` y `Foto` ; `Repositorios` compone los cuatro puertos.
- **Herencia de frameworks (aceptada):** las entidades heredan de `pydantic.BaseModel` a través de `Entidad` .
- **Herencia para reutilizar código (a revisar):**
- `_ConRepositorios` ([postulaciones.py:18](#)) da a los casos de uso de postulaciones los helpers `_animal()` y `_postulacion()` . `Postular` tuvo que sobrescribir el constructor para agregar

`adoptantes` : señal típica de que la herencia empieza a estorbar. Alternativa: funciones auxiliares o un pequeño objeto `Buscador` compuesto.

- `_Almacen` en el adaptador in-memory: herencia genérica para compartir copias e ids; es periferia y solo para pruebas, costo bajo.

STUPID: los anti-principios

Letra	Anti-principio	¿Presente?	Evidencia
S	Singleton	⚠ Controlado	El cliente de Mongo es un global de módulo (<code>mongo/client.py:5</code>) y <code>get_repositorios</code> usa <code>@lru_cache</code> . Ambos viven en adaptadores y se reemplazan en pruebas con <code>dependency_overrides</code> ; el núcleo no los conoce.
T	Acoplamiento fuerte	⚠ Solo en la periferia	El núcleo depende de abstracciones. Excepción: <code>InMemoryPostulacionRepository</code> recibe el <code>InMemoryAnimalRepository</code> concreto para filtrar por refugio (<code>memory/repositories.py:76</code>), y las pruebas HTTP siembran datos con el método privado <code>_guardar_copia</code> .
U	No testeable	✅ Evitado	100 % de cobertura del núcleo, sin red, en ≈ 2 s (evidencia de testabilidad).
P	Optimización prematura	✅ Evitado	Las inclusiones (<code>include=</code>) consultan uno por uno, sin caché ni agregaciones; se optimizará en el adaptador de Mongo si hace falta, sin tocar el núcleo.
I	Nombres poco descriptivos	✅ Evitado	Nombres del lenguaje del negocio: <code>validar_postulable</code> , <code>cerrar_automaticamente</code> , <code>existe_pendiente</code> , <code>PostulacionDuplicada</code> .
D	Duplicación	✅ Evitado	Ver DRY: una tabla de errores, un <code>_resolver()</code> , una base de almacenes.

Conclusión

Hexagonal empuja de forma natural **SOLID completo** —en especial la **D**, que es la definición misma del estilo— y la **segregación de interfaces**, porque cada puerto nace de una necesidad concreta del núcleo. Los principios que el estilo **no garantiza** (KISS, Demeter, composición) dependen de la disciplina dentro de cada capa, y ahí aparecen las excepciones que documentamos: todas están en la **periferia** (adaptadores y pruebas), ninguna en el dominio. Eso es coherente con el objetivo del estilo: proteger las reglas de negocio aunque los bordes sean imperfectos.

Mejoras identificadas (no bloqueantes): 1. Reemplazar la herencia `_ConRepositorios` por composición. 2. Exponer `animal_id()` / `adoptante_id()` en los esquemas para cortar la cadena de accesos del router. 3. Agregar un método público de siembra a los adaptadores in-memory para que las pruebas no usen `_guardar_copia`.

Fuentes

- Martin, R. C. (2017). *Clean Architecture*. Prentice Hall (SOLID y regla de dependencia).

- Cockburn, A. (2005). *Hexagonal Architecture (Ports and Adapters)*.
<https://alistair.cockburn.us/hexagonal-architecture/>
- Lieberherr, K., Holland, I. (1989). *Assuring good style for object-oriented programs* (Ley de Demeter). IEEE Software.
- Gamma, E. et al. (1994). *Design Patterns*. Addison-Wesley ("favorecer la composición sobre la herencia").
- Hunt, A., Thomas, D. (1999). *The Pragmatic Programmer*. Addison-Wesley (DRY).